
Jigsaw: Retrieval-Constrained Exact Subgraph Matching

Proceedings Track Submission

Anonymous Author(s)

Anonymous Affiliation

Anonymous Email

Abstract

Exact subgraph-isomorphism solvers such as Glasgow are sound and complete, but million-node attributed graphs are difficult to serve: continuous features must be mapped to discrete matching labels, and whole-graph residence can exceed memory or leave the verifier an unmanageable search domain. Under our tested configuration, direct CP-based Glasgow solves **0 of 15** sampled queries on full OGBN-MAG (1.9M nodes) and 0/15 on OGBN-Arxiv (169K), while solving 15/15 on Cora (19.8K) in ≈ 2.7 s. **Jigsaw** turns learned graph retrieval into an execution layer for exact search. A GNN trained with **FullCov**, an objective that rewards coverage of every partition touched by a match, ranks hierarchical graph partitions. Jigsaw loads a bounded set, adds only one-hop boundary nodes from neighboring partitions, prunes by typed signatures and exact labels, and gives the resulting candidate to Glasgow. Any accepted match is exact inside that candidate. Because retrieval and verification are separate, Jigsaw can select a learned or classical ranker from measured label selectivity. A classical label index leads under near-unique labels (96.7% versus 88.6% for the learned ranker), while learned retrieval becomes the better partition ranker as labels coarsen. FullCov improves fixed-budget coverage on locked Arxiv K-hop tests (FullCov@50 39.0% $\rightarrow$ 48.8%, FullCov@100 66.0% $\rightarrow$ 82.0%; exact McNemar $p = 9.8 \times 10^{-6}$, 4.0×10^{-10}). Across Cora, Arxiv, and MAG (two seeds, eight families, three sizes, six policies), Jigsaw recovers 88.6% of MAG positives with learned retrieval and 98.4% with exhaustive partition filtering; audited no-match accuracy is 99.3% to 100.0%. At matched half-partition budgets, boundary overlap nearly doubles MAG recovery (44.4% $\rightarrow$ 88.9%), while query-derived pruning provably preserves every covered match. Candidate construction, not exact solving, dominates latency. A two-pass streaming serve keeps only a partition cache resident, using 2.4 GB on MAG instead of 10.2 GB for whole-graph residence ($4.2\times$ less).

1 Introduction

Subgraph isomorphism is a basic tool for motif discovery, scientific graph analysis, fraud search, and security auditing. The exact decision problem is NP-complete [9]; modern constraint solvers such as Glasgow [1] are strong, but large candidate domains and whole-graph residence still make them difficult to serve at scale. Learned retrieval can identify promising graph regions, but it cannot certify a match. Our question is whether retrieval can reduce the target enough for exact verification to fit a practical memory budget. Jigsaw uses the neural model only to choose where to search and leaves acceptance to Glasgow.

We call this design *retrieval-constrained verification*. Jigsaw partitions a large graph, embeds queries and partitions in a shared space, retrieves a bounded set of partitions, stitches their overlap neighborhoods, prunes by typed structural signatures and exact labels, and then calls the verifier. If the candidate contains a valid embedding, Glasgow can certify it exactly. Missing a required region limits global completeness but cannot create a false match. FullCov trains the retriever against that specific failure event by requiring coverage of every partition touched by the planted match.

We evaluate the full execution path across Cora, OGBN-Arxiv, and MAG using two locked seeds, three target sizes, six positive and two negative query families, and six retrieval policies. FullCov-aligned training improves retrieval, while overlap and pruning keep exact verification workable. Cora and Arxiv reach 100% only when every partition is examined. At the comparable half-partition budget, policies separate: overlap-aware training lifts Arxiv from 74% to 93%, Cora remains near saturation at 94%, and the walk-aware MAG model reaches 88.6%. Filtering every partition remains a useful recall ceiling, not a scalable retrieval policy.

Contributions.

- We **scale** exact subgraph matching to million-node graphs via retrieval-constrained verification, making it tractable where a direct full-graph call to the CP-based Glasgow solver solves 0/15 MAG queries (node features are discretized to the integer vertex labels the solver matches on). Because the cascade prunes partition-by-partition, it never holds the whole graph resident: a conservative streaming serve runs at 2.4 GB versus the 10.2 GB a direct solve needs to keep MAG in memory ($4.2\times$).
- We formulate the pipeline as coverage-conditioned exact verification, separating verifier soundness (always guaranteed) from retrieval completeness (a tunable budget knob), and prove the pruning stage is coverage-lossless (Proposition 1).
- We introduce **FullCov**, a coverage-conditioned training objective based on min/CVaR over the weakest required partition. It improves fixed-budget coverage, and walk-aware training beats the matched learned/fusion configuration 92/35 on MAG ($p=4.4\times 10^{-7}$), with gains concentrated in random-walk and multi-coarse. Several policies localize well on label-rich benchmarks, so the learned advantage appears where labels no longer localize the query reliably.
- We provide a two-seed, three-dataset benchmark with positive and negative query families, exact-verification metrics, and an audited evaluation. We measure each candidate-shrinkage stage and show that boundary overlap is essential for recall while query-derived pruning preserves every covered match.

2 Related Work

Exact subgraph search dates to Ullmann [10]; modern systems such as VF2 [5], RI [11], LAD [12], VF3 [22], and Glasgow [1] combine propagation, branching, and search ordering. In-memory *filter-and-verify* matchers include GQL [21], TurboISO [18], CFL-Match [19], DAF/DP-iso [20], and GSI [23]; comparative studies chart their filtering, ordering, and enumeration tradeoffs [24, 25]. These methods build query-vertex candidates over a resident graph. Relational engines instead compile patterns into binary or worst-case-optimal joins [26], including distributed low-memory dataflows [27]. Out-of-core enumeration (DUALSIM [28]) and distributed systems (STwig/Trinity [29], G-thinker [30]) stream or shard the *entire* graph. Jigsaw contributes a complementary single-machine point: learned relevance determines which regions enter memory under an explicit budget. Learned graph matching builds on expressive GNN encoders such as GIN [2] and spans graph-pair similarity models [31, 32], partitioned similarity estimation [33], and subgraph-retrieval models such as ISONET and ISONET++ [34, 35]; a recent design-space study unifies their alignment choices [36]. These methods score graph pairs or corpus graphs rather than certify a bounded region of one large target. NeuroMatch [14], GLSearch [16], and neural counting [17] likewise predict containment, alignments, searches, or counts. GNN-PE [15] couples learned path-dominance pruning to exact matching but retains a whole-graph index, while NeuGN [37] learns navigation inside a resident exact search. Jigsaw instead learns the memory boundary, ranks METIS [7] regions with FAISS [6], and leaves acceptance to Glasgow. Our CFL/DP-iso/GQL benchmark shows why that boundary matters: resident methods finish 27/27 high-selectivity MAG executions but only 12/27 under coarse labels; full-graph Glasgow finishes 0/15.

Low-selectivity diagnostic. With MAG labels coarsened to 32 deterministic buckets, the three filter-and-verify matchers complete only 12/27 executions: 8/9 at size 20 but 2/9 at each of sizes 50 and 100, with random-walk worst at 3/9. Candidate tables reach 2.4M entries and 15 calls hit the 120 s limit, locating the failure in whole-graph candidate construction as label selectivity falls rather than in high-selectivity enumeration.

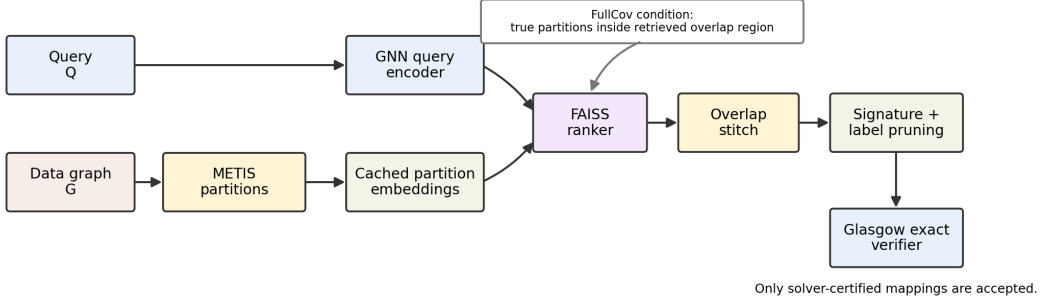


Figure 1: Jigsaw pipeline. The learned component ranks partitions, but the only accepting step is Glasgow verification inside the stitched candidate graph. FullCov is the condition under which the candidate contains every partition needed by the planted match.

3 Problem and Guarantee

Let $G = (V, E, X)$ be a large attributed graph and $Q = (V_Q, E_Q, X_Q)$ a query. The full exact task asks whether Q is isomorphic to an induced or non-induced subgraph of G under the solver’s configured semantics. Jigsaw instead builds a candidate graph $G_C \subseteq G$ and runs the exact verifier on G_C . The verifier is sound inside G_C : a returned match is a true match. Completeness is conditioned on coverage. Let $\mathcal{T}(Q)$ be the set of coarse partitions touched by the planted match and $R_K(Q)$ the retrieved top- K partition set. We define

$$\text{FullCov@K}(Q) = \mathbf{1}[\mathcal{T}(Q) \subseteq R_K(Q)]. \quad (1)$$

For overlap-cascade verification, R_K is expanded by one-hop partition overlap before pruning. FullCov is a sufficient retrieval condition for exact verification over planted positives; negative queries instead test whether the candidate-and-solver pipeline correctly returns no match.

Budget as a soundness-preserving completeness dial. Soundness does not depend on K : any returned match is valid at every budget. The budget controls completeness. At $K = |\mathcal{P}|$ (all partitions; FilterAll), retrieval covers the complete partition set and matching-invariant pruning preserves every valid embedding, so verification is complete under the configured label semantics whenever the verifier finishes. Smaller budgets expose an explicit recall-cost tradeoff. The solved-by-budget curves in Appendix D measure the realized completeness under the timeout, allowing a caller to choose between exhaustive search and a quantified miss or timeout rate.

4 Jigsaw Method

The data graph is partitioned into fine and coarse METIS blocks; coarse blocks are the retrieval units. For each query, Jigsaw computes an embedding $z_Q = f_\theta(Q)$ and ranks all coarse partitions by cosine similarity to cached partition embeddings, with FAISS providing fast top- K retrieval (the encoder architecture is detailed in Appendix B). Hard partition boundaries are brittle: true query neighborhoods often cross coarse blocks, so Jigsaw stitches the retrieved partitions plus one-hop neighboring overlap, increasing recall before verification while still avoiding full-graph search. The stitched region is then filtered by typed structural signatures and exact labels; signatures match coarse type and feature fingerprints, and exact labels are especially important for negative-label queries. The final answer is produced only by the exact verifier.

FullCov objective. Standard contrastive training rewards high average similarity to positives, which can still leave one required partition ranked too low. Jigsaw optimizes the bottleneck. For each query Q_i , let $\mathcal{P}_i$ be its positive partitions and let s_{ij} be the query-partition score. The objective combines multi-positive contrastive learning with a weak-positive term:

$$\mathcal{L}_{\text{weak}}(i) = -\log \frac{\exp(\min_{p \in \mathcal{P}_i} s_{ip}/\tau)}{\exp(\min_{p \in \mathcal{P}_i} s_{ip}/\tau) + \sum_{n \in \mathcal{N}_i} \exp(s_{in}/\tau)}. \quad (2)$$

Additional CVaR and barrier terms penalize positives that fall below the chosen retrieval budget. Partition embeddings are refreshed from cache, with live re-encoding of required positives so the weakest positive receives a usable gradient.

Why the weakest positive. The choice of min (and its CVaR relaxation) follows directly from the metric. $\text{FullCov@K}(Q) = \mathbf{1}[\mathcal{T}(Q) \subseteq R_K(Q)]$ is a *conjunction*: it succeeds only if *every* $p \in \mathcal{P}_i$ is ranked within K , so failure is governed by the single worst-ranked required partition, $\min_{p \in \mathcal{P}_i} s_{ip}$, not the average positive score standard contrastive learning maximizes. Optimizing the average can raise mean recall while leaving one required partition below the budget. Exact verification cannot tolerate that failure. CVaR (the mean of the worst α -fraction of positives) is a smooth surrogate for the non-differentiable min that spreads gradient over the few hardest required partitions; this is why, in Table 1, the objective improves fixed-budget FullCov most at the operating points the cascade uses, even though average recall barely moves. The retriever only proposes a candidate boundary; once G_C is constructed, Glasgow gives exact answers. A positive can fail because the boundary misses the planted solution or because the candidate grows too large and the solver times out. The benchmark records both cases.

5 Experimental Protocol

Datasets. We evaluate Cora [13], OGBN-Arxiv [8], and the full heterogeneous OGBN-MAG. Cora is small enough for exact timing sanity checks; Arxiv is the main medium-scale benchmark; OGBN-MAG is a large heterogeneous graph (1,939,743 nodes across four node types joined by typed relations) and stresses the approach with both scale and relation-aware structure. It is modeled relation-aware (RGCN over retained node/edge types), not collapsed to an attribute-only homogeneous graph.

Label granularity. The exact verifier matches on discrete vertex labels, the matching primitive of classical label-based graph indexing [38, 39]. Here they are obtained by hashing each node’s full attribute vector ($\sigma(v) = \text{md5}(x_v) \bmod 10^6$) and are highly but not perfectly discriminative: the 10^6 codebook admits collisions across the $\sim 736\text{K}$ featured MAG nodes, so a returned match is exact w.r.t. *hashed-label* semantics rather than raw attribute equality (the one benchmark-level false positive per MAG method). At serving time, label and signature tokens are derived exclusively from the submitted query payload; planted target-node IDs are retained only as evaluation metadata and never enter candidate construction. These highly selective labels are the dominant reason every policy recovers all Cora and Arxiv positives once coverage is met. We call these *label-rich* benchmarks and treat retrieval as a candidate-cost rather than accuracy lever. The coarsened-label experiments weaken pruning and make retrieval more decisive, isolating the label-selectivity regime where learned localization contributes.

Queries and retrieval policies. The production matrix uses two query seeds, three target sizes (20/50/100), and 50 queries per type/size/seed. Positive families are single-partition, multi-fine, K-hop, degree K-hop, random walk, and multi-coarse; negatives are negative-label and negative-structure. Every multi-region positive is a connected planted subgraph. Six policies isolate the main retrieval mechanisms: **Jigsaw** (learned GraphSAGE/RGCN partition ranker), **MeanFeat** (cosine on mean node features), **Mean-RRF** (reciprocal-rank fusion of the learned Jigsaw and MeanFeat rankings, which is a configuration of our own retriever rather than a nonparametric baseline), **TopoFeat** (coarse structural descriptors), **Random** (uninformed control), and **FilterAll** (all partitions to pruning/verification, serving as a recall ceiling rather than a scalable retriever). A seventh policy, **FeatureIndex**, is a classical label-coverage index in the GraphGrep [38]/gIndex [39] tradition that ranks partitions by how many of the query’s exact labels they contain. We evaluate it on MAG to identify the label-selectivity crossover between classical and learned localization.

Partition and overlap scale. The partitions are balanced (MAG: 2,000 coarse partitions of median size 969, 10,000 fine partitions of median size 194). For a selected partition, the overlap operator adds only the *boundary nodes* of partitions sharing an edge with it. This is a tight per-partition expansion ($5.4\times$ on MAG), far below the loose whole-neighbor-union bound it never realizes (Appendix C). Breadth arises only when the cascade unions these overlaps across the retrieval budget; the verifier sees the much smaller pruned candidate of Table 2. Appendix C (Tables 4, 5) gives the full scale, and Section 6.3 dissects how much each cascade stage removes.

Metrics. We report FullCov@K and recall for retrieval ablations, and positive exact-solve rate, no-match accuracy, false positives, timeouts, candidate size, and wall-clock components for exact verification. Each dataset has exactly 2,400 unique queries: 1,800 positives (6 families) and 600

Table 1: Matched Arxiv retrieval objective ablation. Fixed retrieval budgets on $n=400$ locked K-hop queries (200 per query seed); each row averages the validation-selected checkpoint from two independent training replicates.

Objective	FullCov@20	FullCov@50	FullCov@100	Mean worst rank
Contrastive control	21.7%	39.0%	66.0%	74.02
FullCov objective	26.0%	48.8%	82.0%	58.97
McNemar p	4.6×10^{-3}	9.8×10^{-6}	4.0×10^{-10}	—

negatives (2 families), from 2 seeds, 3 sizes, and 50 queries per cell. Applying 6 policies yields 14,400 policy-query evaluations per dataset before budget expansion; these are evaluation rows, not additional unique queries. Equivalently, each dataset has 288 production summary rows (6 methods, 2 seeds, 8 families, 3 sizes). A serving-path audit covers all 7,200 unique queries: query-payload tokens equal the legacy evaluation tokens for all 5,400 positives, all 900 label-corrupted negatives intentionally diverge, and all 1,800 negatives were rerun with pruning tokens derived only from the submitted query. Diagnostic variants are preserved separately without inflating the main grid.

Implementation and reproducibility. Partitioning uses METIS; partition and query embeddings are indexed with FAISS; exact verification uses the Glasgow Subgraph Solver on integer-labeled candidate graphs obtained by discretizing node features. All reported timings are per-query wall-clock on 8-vCPU Google Cloud instances, with the exact verifier under a 30 s per-query budget. We will release the trained encoders, query seeds, partition hierarchies, and checksum-verified benchmark outputs. An anonymized implementation is available for review at <https://anonymous.4open.science/r/Jigsaw>.

6 Results

6.1 FullCov Training Improves Retrieval

Table 1 (visualized in Figure 4) gives the matched-budget Arxiv retrieval result. All rows use the same locked K-hop query seeds and 9,000 optimizer steps. The FullCov objective improves the fixed-budget operating points most relevant to bounded exact verification.

The locked K-hop suite contains no queries that are impossible at $K = 20, 50, 100$; the mean true coarse footprint is 6.37 and the maximum is 17. The remaining misses are ranking errors, and FullCov targets them by optimizing the weakest required partition. Both models use the same query seeds and training budget, ruling out easier queries or longer optimization as explanations for the gain. The largest improvement appears at FullCov@100, the cascade’s operating point, where better ordering matters but the verifier still cannot search the whole graph.

6.2 Three-Dataset Production Matrix

Table 2 reports the cleaned production metrics. Positive queries measure exact planted-match recovery; negative queries measure correct no-match behavior. The table also includes false positives, solver timeouts, average candidate nodes, cascade time, and verifier time. Cascade time sums candidate construction and exact verification; it excludes retrieval/ranking, whose index is built offline and whose implementation-specific latency is reported separately where measured. All values are reproducible from the released benchmark summaries.

Cora and Arxiv reach 100% only at the exhaustive all-partition budget (their 20/200 partitions), so those values are FilterAll ceilings rather than retrieval results. At the comparable half-partition budget, the overlap-aware GraphSAGE retriever solves 94.2% on Cora and 92.8% on Arxiv. Random and topology policies fall between 31.9% and 39.2% on Cora/Arxiv, and between 37% and 51% on MAG. Overlap-aware training lifts Arxiv from 74.1% to 92.8% at this budget, while Cora is already near saturation (95.4% $\rightarrow$ 94.2%). Mean-RRF is close at 93.8%/94.0%, and a classical label index leads MAG under near-unique labels (96.7%). Together, FullCov and the coarsened-label experiments identify the regime where learned localization contributes most.

Table 2: Final production metrics. Pos. uses matched half-partition budgets $K=10/100/1000$ (Cora/Arxiv/MAG); FilterAll uses the explicit all-partition ceiling. Neg. is audited no-match accuracy; FP counts negative false positives, while TO counts positive-query solver timeouts (FeatureIndex was run on positives only). Candidate and timing columns use the same endpoint as each Pos. entry. Cascade time is candidate construction plus exact verification and excludes retrieval/ranking.

Dataset	Method	Pos. %	Neg. %	FP	TO	Cand. nodes	Cascade s	Solver ms
Cora	Jigsaw	94.2	100.0	0	0	458	0.05	11.8
Cora	Mean-RRF	93.8	100.0	0	0	485	0.05	11.8
Cora	MeanFeat	87.7	100.0	0	0	1.13K	0.06	10.5
Cora	TopoFeat	31.9	100.0	0	0	7.60K	0.17	3.6
Cora	Random	32.7	100.0	0	0	7.54K	0.16	3.8
Cora	FilterAll [‡]	100.0	100.0	0	0	59	0.06	12.7
Arxiv	Jigsaw	92.8	100.0	0	0	77	0.19	18.3
Arxiv	Mean-RRF	94.0	100.0	0	0	75	0.19	18.8
Arxiv	MeanFeat	89.7	100.0	0	0	91	0.22	18.4
Arxiv	TopoFeat	33.3	100.0	0	0	233	0.46	6.0
Arxiv	Random	39.2	100.0	0	0	219	0.47	7.0
Arxiv	FilterAll [‡]	100.0	100.0	0	0	57	0.35	19.1
MAG	Jigsaw	88.6	99.5	1	25	138.1K	6.97	81.9
MAG	Mean-RRF	85.4	99.5	1	17	137.1K	5.38	67.9
MAG	FeatureIndex [§]	96.7	—	—	24	59.4K	2.39	118.8
MAG	MeanFeat	64.6	99.5	1	12	212.1K	8.02	27.5
MAG	TopoFeat	37.1	99.8	1	1	235.6K	8.07	5.2
MAG	Random	51.4	99.7	1	9	282.1K	9.54	19.9
MAG	FilterAll [‡]	98.4	99.3	1	33	443.8K	5.85	288.6

MAG is the stress case: random and topology degrade sharply, mean features help, and the deployed walk-aware model reaches **88.6%** while sending the verifier $3.2\times$ fewer candidate nodes than FilterAll (138K vs. 444K). Difficulty grows with query footprint rather than node count alone. Learned-Jigsaw recovery declines from 93.2% at size 20 to 90.7% at 50 and 82.0% at 100, driven by random-walk (94/100, 79/100, 40/100). Contained multi-fine queries remain 100/100 at every size, and multi-coarse holds at 77/100, 79/100, and 72/100. FilterAll reaches 98.4%, but it gives the verifier all partitions and serves as a recall ceiling rather than a scalable policy. **The cascade never holds the whole graph resident:** it admits partitions one at a time and prunes with query-payload-derived structural-signature and exact-label sets before materializing edges. A conservative streaming serve (resident cache of eight partitions, survivor edges fetched in a second pass) peaks at **2.4 GB** across a 54-query MAG grid (6 families $\times$ 3 sizes $\times$ 3 queries), compared with **10.2 GB** to hold MAG for a direct solve. This is a $4.2\times$ reduction. The serve recovers 46/54 planted matches (all easy-family queries; the 8 misses are random-walk/multi-coarse) and solves components with a median 110 nodes (maximum 9,497) in 5 ms to 8.4 s.

Label-selectivity crossover. Near-unique md5 labels give exact-label methods unusually strong signal. In that setting, the classical label-coverage index (**FeatureIndex**, GraphGrep/gIndex-style) is the strongest bounded retriever (96.7% solve, median worst-true-partition rank 104 of 2,000), ahead of the learned GNN (88.6%; median rank $\sim$ 900). Once labels are coarsened, the ordering reverses. The learned ranker beats the label index in median worst-true-partition rank (5 vs. 7 on Cora and 44 vs. 104 on Arxiv at class labels; Table 8) and leads in every MAG query family at coarse 39-class labels. Complete coarse-label solve runs are statistically tied (FeatureIndex 63.4%; learned variants between 63% and 64%). The validated learning result is therefore localization: the GNN becomes the better partition ranker after exact labels lose selectivity.

Spatial footprint explains the remaining retrieval gap. The learned ranker localizes contained queries almost perfectly (single/multi-fine ranks from 3 to 7), while diffuse families require coverage across many distant partitions. The inference-time variants in Appendix G leave FullCov@1000 within ≈ 2 points, locating the bottleneck in the single-vector representation rather than score aggregation. An offline label-selectivity statistic chooses between the label index and learned ranker without changing the downstream cascade. Within the learned policy, FullCov and aligned training improve random-walk and multi-coarse coverage and lift Arxiv from 74.1% to 92.8%. The systems result is memory-bounded exact verification, and the learning result is improved coverage where label indexing no longer localizes reliably.

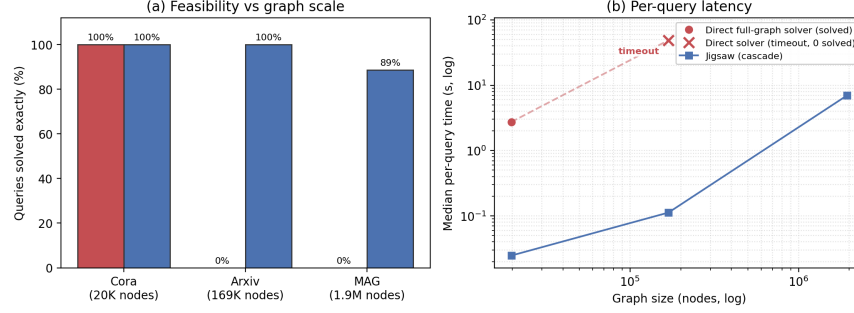


Figure 2: Jigsaw makes the exact solver’s domain tractable. Cora/Arxiv use the same 15 K-hop queries at matched half-partition budgets ($K = 10/100$), where Jigsaw solves 15/15 on both; the MAG cascade point is the separate 1,800-positive production mean at $K = 1,000$. Direct Arxiv is timeout-censored in the latency panel; the retained MAG diagnostic records the timeout-censored 0/15 outcome but no comparable wall-clock point.

Table 3: Scaling and latency: direct full-graph solver vs. Jigsaw cascade. Cora/Arxiv use the same 15 K-hop queries and Jigsaw is evaluated at matched half-partition budgets ($K = 10/100$), avoiding exhaustive Cora retrieval. The MAG direct cell uses its 15-query timeout diagnostic, while * the MAG Jigsaw cell is separately the 1,800-positive production mean at $K = 1,000$. † Glasgow’s 30 s Arxiv budget may be exceeded during cleanup/loading.

Dataset	Nodes	Direct full-graph solver		Jigsaw cascade		Cand. nodes
		Solved	Med. time	Solved	Med. time	
Cora	19.8K	15/15	2.74 s	15/15	0.03 s	50
Arxiv	169K	0/15 †	≈49 s	15/15	0.11 s	50
MAG	1.9M	0/15	timeout	88.6%	6.97 s*	—

254 Direct Glasgow provides the CP-verifier control. Full-graph matching with no retrieval solves **0 of 15**
 255 sampled MAG K-hop queries, with no query returning in the timeout-censored diagnostic. Under
 256 this resource budget, retrieval is necessary for Glasgow rather than a minor optimization. The same
 257 configuration solves all 15 Cora queries but **0 of 15** on OGBN-Arxiv (169K nodes), while Jigsaw
 258 remains feasible by handing Glasgow a ~ 50 -node pruned candidate (Table 3, Figure 2). This places
 259 the tested crossover between Cora and Arxiv. Resident filter-and-verify matchers can be faster after
 260 loading when labels are selective; Section 7 scopes our claim to Glasgow, bounded-memory service,
 261 and the low-selectivity regime. Mean-RRF (85.4%) fuses the learned and mean-feature rankings
 262 within Jigsaw, and the pure learned ranker leads it by 3.2 points. Both exceed the mean-feature
 263 baseline (64.6%) by more than 20 points. These configurations occupy different points on the same
 264 recall-cost frontier (Appendix E).

265 Bootstrap 95% confidence intervals and paired exact-McNemar tests on the MAG solve rate confirm
 266 this ordering among the interchangeable retrieval *components* (full statistics, per-family solve rates,
 267 and budget curves in Appendices E and D). These comparisons evaluate retrieval components within
 268 the same surrounding system. That system enables exact matching at scale without holding the graph
 269 resident, while FullCov secures coverage on the hard families.

270 6.3 Candidate-Shrinkage Cascade and Boundary Overlap

271 All reported production solve rates use one-hop boundary overlap: selected partitions admit boundary
 272 nodes from adjacent partitions, not the neighboring partitions in full. To test whether unioning this
 273 boundary overlap makes partitioning irrelevant, we measure the per-query cascade full graph $\rightarrow$
 274 overlap $\rightarrow$ signature $\rightarrow$ label-pruned $\rightarrow$ solver on 14,400 positive method/model-query evaluation
 275 records over the 1,800-query MAG positive grid from the generic-mix-encoder run at the decided
 276 budget. Hybrid and Mean-RRF each contribute two model variants, so 14,400 is an evaluation-record
 277 count rather than a unique-query denominator. Three facts emerge. **(1) The union is broad, but**
 278 **ranking still removes most of the graph:** the median overlap union is 59% of MAG for Jigsaw,

compared with 73% to 83% for mean-feature, random, and topology retrieval. After identical pruning, Jigsaw’s median candidate has 84.5K nodes versus 544K for FilterAll (the production matrix reports means, 138.1K vs. 443.8K). Ranking alone accounts for this $6.4\times$ reduction.

(2) All recall loss occurs at the overlap stage; pruning is coverage-lossless. This follows from the pruning operator, not only from the measurements.

Proposition 1 (Lossless pruning). *Let the pruning token $\sigma(\cdot)$ be a node attribute that the matching semantics preserve, i.e. any valid embedding ϕ of Q into G satisfies $\sigma(\phi(q)) = \sigma(q)$ for every query node q (vertex-labelled matching, with σ derived from node type and feature labels). Prune the candidate G_C to $G'_C = \{v \in G_C : \sigma(v) \in \sigma(V_Q)\}$. Then for any embedding ϕ with $\phi(V_Q) \subseteq G_C$, we have $\phi(V_Q) \subseteq G'_C$. Hence pruning preserves node coverage: FullCov is unchanged.*

Proof. For $q \in V_Q$, $\sigma(\phi(q)) = \sigma(q) \in \sigma(V_Q)$, so $\phi(q)$ satisfies the keep condition and is retained. Thus every matched node survives.

The cascade signatures (node type $\times$ feature-bit hashes) and the exact label filter are such attribute invariants, so Proposition 1 applies; a *degree*-based token would not be matching-invariant ($\deg(\phi(q)) \geq \deg(q)$ only), which is why the pipeline prunes on attributes, not degree. Empirically, for every policy the planted-match survival rate after overlap equals that after pruning (Jigsaw 87% both, in this generic-encoder diagnostic whose end-to-end solve is 86.0%), so the solve ceiling is set entirely by whether the retrieved-plus-overlapped region *contains* the match.

(3) Latency is dominated by candidate construction, not solving (per-query build $\gg$ solve; Appendix E): the verifier is cheap, and assembling the broad overlap union is the cost.

Boundary overlap is an essential recall operator. The matched operator ablation shows its effect directly: removing boundary overlap cuts MAG solve rate from 88.9% to 44.4% and Arxiv from 94.4% to 86.1% (Appendix F). Its benefit is strongest for compact local structure. When retrieval misses a true partition, one-hop boundary overlap recovers it for 94% of K-hop queries but for 62% of random-walk queries, whose matches more often extend into the interior of an unselected partition. This family dependence explains why overlap sharply raises overall recovery while diffuse random-walk queries remain the hardest case. Together with the lossless-pruning result above, the measurements locate the remaining errors before verification: a query fails only when retrieval plus boundary overlap does not cover its match.

6.4 Negative Queries and Soundness

Jigsaw only accepts a match after Glasgow verification, so negative-label and negative-structure queries test the no-match behavior of candidate construction and the verifier. Audited no-match accuracy is 100.0% on Cora/Arxiv and between 99.3% and 99.8% across MAG policies, with at most one false positive per method; the remaining errors are timeout/unknown outcomes. That false positive is a real isomorphism that the perturbed query happens to admit elsewhere in the graph, not a fabricated answer. The reported metric is audited no-match accuracy under the synthetic negative protocol; global non-existence would require exhaustive full-graph certification.

7 Discussion and Limitations

Jigsaw preserves exact verification while using learned retrieval to choose the search boundary. If retrieval misses a required region, the solver cannot recover that match; FullCov turns this source of incompleteness into a measurable training and serving objective. The design targets static graphs with repeated queries, where partitioning, embedding, and FAISS indexing amortize. Cora and Arxiv expose saturation under selective labels, while MAG separates the policies: FeatureIndex leads with near-unique labels and learned retrieval leads after labels are coarsened. FilterAll remains the exhaustive upper bound against which bounded retrieval is measured.

Evaluation scope. We compare against classical feature/structure-index rankers (MeanFeat/TopoFeat, GraphGrep [38]/gIndex [39]-style filters), a no-retrieval exact control (full-graph Glasgow, 0/15 MAG), an exhaustive FilterAll upper bound, and uninformed retrievers. We also benchmark scalable filter-and-verify matchers (CFL-Match, DP-iso, GQL): they solve Cora/Arxiv and high-selectivity MAG once loaded, while completing only 12/27 coarse-label MAG solver-query executions. Together these controls locate Jigsaw’s advantage in the CP-based Glasgow path,

bounded-memory service, and low-selectivity candidate construction. NeuroMatch and GLSearch use released protocols for smaller, contained-query targets rather than a partitioned million-node execution path. Out-of-core and distributed systems (DUALSIM [28], STwig/Trinity [29], G-thinker [30]) address residence by streaming or sharding the entire graph; Jigsaw instead uses learned region relevance to expose a tunable single-machine memory frontier. Its one-time partitioning, training, and indexing costs amortize over repeated queries, making the design especially useful for graphs that cannot remain resident and for multi-tenant or cold-start service. The negative families are audited synthetic no-match tests, and the positives are planted connected subgraphs on static graphs. We therefore report no-match accuracy rather than global negative completeness and separate bounded retrieval from the FilterAll ceiling. Within this scope, FullCov improves hard-family coverage and the complete cascade turns the tested Glasgow operating point from 0/15 direct solves into 88.6% exact positive recovery.

The GNN-PE public-release feasibility diagnostic is reported in Appendix A.

8 Conclusion

Jigsaw turns learned graph retrieval into an execution mechanism for exact search. The model decides which regions deserve memory and verification; Glasgow remains the only component allowed to accept a match. On full MAG, this changes the tested operating point from 0/15 direct CP solves to 88.6% exact positive recovery with the learned retriever, while streaming at 2.4 GB instead of 10.2 GB for whole-graph residence. FullCov trains against the system’s decisive failure event: missing even one required partition. Boundary overlap recovers cross-partition matches, and lossless query-derived pruning bounds the verifier domain. Two operating regimes appear in the data. Selective labels favor resident exact methods because their candidate tables collapse; weak labels and bounded residency make whole-graph candidate construction fail. Jigsaw remains feasible in the latter regime by selecting a label index or learned ranker from measured label selectivity. This adaptive portfolio uses classical indexing when exact labels localize well and learned ranking when they do not; FullCov further improves the hard families. Jigsaw makes exact verification usable on million-node static graphs under a measurable memory budget without replacing exactness with a neural guess.

References

- [1] McCreesh, C., Prosser, P., Trimble, J.: The Glasgow Subgraph Solver: Using Constraint Programming to Tackle Hard Subgraph Isomorphism Problem Variants. In: CP 2020, LNCS, vol. 12333, pp. 316–333. Springer (2020) 1, 2
- [2] Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How Powerful are Graph Neural Networks? In: ICLR (2019) 2
- [3] Hamilton, W.L., Ying, R., Leskovec, J.: Inductive Representation Learning on Large Graphs. In: NeurIPS, pp. 1024–1034 (2017) 11
- [4] Schlichtkrull, M., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling Relational Data with Graph Convolutional Networks. In: ESWC, pp. 593–607 (2018) 11
- [5] Cordella, L.P., Foggia, P., Sansone, C., Vento, M.: A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs. IEEE TPAMI 26(10), 1367–1372 (2004) 2
- [6] Johnson, J., Douze, M., Jégou, H.: Billion-Scale Similarity Search with GPUs. IEEE Trans. Big Data 7(2), 535–547 (2021) 2
- [7] Karypis, G., Kumar, V.: A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. SIAM J. Sci. Comput. 20(1), 359–392 (1998) 2
- [8] Hu, W., et al.: Open Graph Benchmark: Datasets for Machine Learning on Graphs. In: NeurIPS (2020) 4
- [9] Garey, M.R., Johnson, D.S.: Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman (1979) 1
- [10] Ullmann, J.R.: An Algorithm for Subgraph Isomorphism. J. ACM 23(1), 31–42 (1976) 2
- [11] Bonnici, V., Giugno, R., Pulvirenti, A., Shasha, D., Ferro, A.: A Subgraph Isomorphism Algorithm and Its Application to Biochemical Data. BMC Bioinformatics 14(Suppl 7), S13 (2013) 2
- [12] Solnon, C.: AllDifferent-Based Filtering for Subgraph Isomorphism. Artificial Intelligence 174(12–13), 850–864 (2010) 2
- [13] Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., Eliassi-Rad, T.: Collective Classification in Network Data. AI Magazine 29(3), 93–106 (2008) 4

- [14] Ying, R., et al.: Neural Subgraph Matching. arXiv:2007.03092 (2020) 2
- [15] Ye, Y., Lian, X., Chen, M.: Efficient Exact Subgraph Matching via GNN-based Path Dominance Embedding. *Proc. VLDB Endow.* **17**(7), 1628–1641 (2024) 2
- [16] Liu, Y., et al.: GLSearch: Maximum Common Subgraph Detection via Learning to Search. In: *ICML* (2021) 2
- [17] Liu, X., Pan, H., He, M., Song, Y., Jiang, X., Shang, L.: Neural Subgraph Isomorphism Counting. In: *KDD* (2020) 2
- [18] Han, W.S., Lee, J., Lee, J.H.: TurboISO: Towards Ultrafast and Robust Subgraph Isomorphism Search in Large Graph Databases. In: *SIGMOD*, pp. 337–348 (2013) 2
- [19] Bi, F., Chang, L., Lin, X., Qin, L., Zhang, W.: Efficient Subgraph Matching by Postponing Cartesian Products. In: *SIGMOD*, pp. 1199–1214 (2016) 2
- [20] Han, M., Kim, H., Gu, G., Park, K., Han, W.S.: Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In: *SIGMOD*, pp. 1429–1446 (2019) 2
- [21] He, H., Singh, A.K.: Graphs-at-a-Time: Query Language and Access Methods for Graph Databases. In: *SIGMOD*, pp. 405–418 (2008) 2
- [22] Carletti, V., Foggia, P., Saggese, A., Vento, M.: Challenging the Time Complexity of Exact Subgraph Isomorphism for Huge and Dense Graphs with VF3. *IEEE Trans. Pattern Anal. Mach. Intell.* **40**(4), 804–818 (2018) 2
- [23] Zeng, L., Zou, L., Özsu, M.T., Hu, L., Zhang, F.: GSI: GPU-friendly Subgraph Isomorphism. In: *ICDE*, pp. 1249–1260 (2020) 2
- [24] Sun, S., Luo, Q.: In-Memory Subgraph Matching: An In-depth Study. In: *SIGMOD*, pp. 1083–1098 (2020) 2
- [25] Zhang, Z., Lu, Y., Zheng, W., Lin, X.: A Comprehensive Survey and Experimental Study of Subgraph Matching: Trends, Unbiasedness, and Interaction. *Proc. ACM Manag. Data* **2**(1), Article 60 (2024) 2
- [26] Mhedhbi, A., Salihoglu, S.: Optimizing Subgraph Queries by Combining Binary and Worst-Case Optimal Joins. *Proc. VLDB Endow.* **12**(11), 1692–1704 (2019) 2
- [27] Ammar, K., McSherry, F., Salihoglu, S., Joglekar, M.: Distributed Evaluation of Subgraph Queries Using Worst-Case Optimal Low-Memory Dataflows. *Proc. VLDB Endow.* **11**(6), 691–704 (2018) 2
- [28] Kim, H., Lee, J., Bhowmick, S.S., Han, W.-S., Lee, J., Ko, S., Jarrah, M.H.A.: DUALSIM: Parallel Subgraph Enumeration in a Massive Graph on a Single Machine. In: *SIGMOD*, pp. 1231–1245 (2016) 2, 9
- [29] Sun, Z., Wang, H., Wang, H., Shao, B., Li, J.: Efficient Subgraph Matching on Billion Node Graphs. *Proc. VLDB Endow.* **5**(9), 788–799 (2012) 2, 9
- [30] Yan, D., Guo, G., Chowdhury, M.M.R., Özsu, M.T., Ku, W.-S., Lui, J.C.S.: G-thinker: A Distributed Framework for Finding Qualified Subgraphs in a Big Graph with Load Balancing. In: *ICDE* (2020) 2, 9
- [31] Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., Wang, W.: SimGNN: A Neural Network Approach to Fast Graph Similarity Computation. In: *WSDM*, pp. 384–392 (2019) 2
- [32] Li, Y., Gu, C., Dullien, T., Vinyals, O., Kohli, P.: Graph Matching Networks for Learning the Similarity of Graph Structured Objects. In: *ICML, PMLR* **97**, 3835–3845 (2019) 2
- [33] Xu, H., Duan, Z., Wang, Y., Feng, J., Chen, R., Zhang, Q., Xu, Z.: Graph Partitioning and Graph Neural Network Based Hierarchical Graph Matching for Graph Similarity Computation. *Neurocomputing* **439**, 348–362 (2021) 2
- [34] Roy, I., Velugoti, V.S.B.R., Chakrabarti, S., De, A.: Interpretable Neural Subgraph Matching for Graph Retrieval. In: *AAAI* **36**(7), 8115–8123 (2022) 2
- [35] Ramachandran, A., Raj, V., Roy, I., Chakrabarti, S., De, A.: Iteratively Refined Early Interaction Alignment for Subgraph Matching Based Graph Retrieval. In: *NeurIPS* **37** (2024) 2
- [36] Raj, V., Roy, I., Ramachandran, A., Chakrabarti, S., De, A.: Charting the Design Space of Neural Graph Representations for Subgraph Matching. In: *ICLR* (2025) 2
- [37] Ying, Y., Dai, Y., Li, W., et al.: Neural Graph Navigation for Intelligent Subgraph Matching. In: *AAAI* **40**(19), 16181–16189 (2026) 2
- [38] Shasha, D., Wang, J.T.L., Giugno, R.: Algorithmics and Applications of Tree and Graph Searching. In: *PODS*, pp. 39–52 (2002) 4, 8
- [39] Yan, X., Yu, P.S., Han, J.: Graph Indexing: A Frequent Structure-based Approach. In: *SIGMOD*, pp. 335–346 (2004) 4, 8

A GNN-PE Public-Release Diagnostic

Beyond the internal policies, our external baselines include the direct-solver control (Table 3, 0/15 on Arxiv/MAG) and a GNN-PE public-release diagnostic. On Cora the released pipeline passes its self-test but, after class-label export and build repairs, recovers 7/24 planted positives at $e = 4$ (6 diagnostic families $\times$ 4 sizes $\times$ 1 query; 0/6 at size 100). The release exposes an inference path but no supervised training recipe for this protocol, so the result measures public-release transfer rather than a retrained reproduction.

B Encoder Architecture and Training Interface

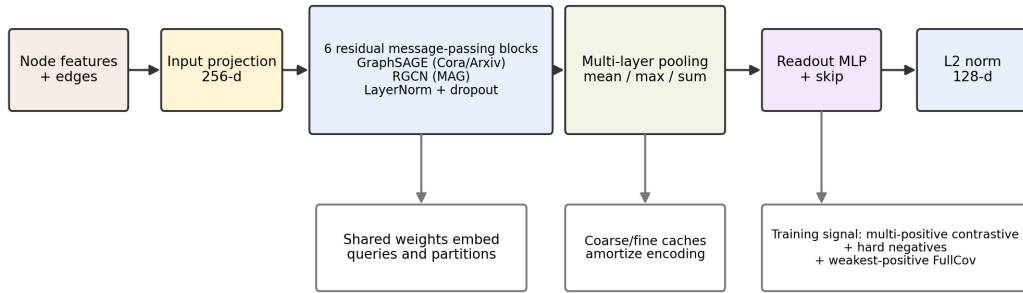


Figure 3: Retrieval encoder. Cora and Arxiv use a six-block residual GraphSAGE encoder; MAG uses the same six-block residual architecture with relation-aware RGCN layers. The same encoder embeds queries and cached graph partitions into a normalized 128-dimensional space.

The encoder is intentionally shared between query graphs and partition graphs. Input node features are projected to a 256-dimensional hidden space, passed through six residual message-passing blocks, normalized with LayerNorm, and read out with multi-layer global mean, max, and sum pooling. The readout is an MLP plus a skip projection, followed by L2 normalization to produce a 128-dimensional retrieval vector. This matters because the retrieval problem is not node alignment; it is region localization. The model must place a query near every partition needed for exact verification, including the weakest or farthest true partition.

For Cora and Arxiv, the message-passing block is GraphSAGE [3] (SAGEConv); on these label-rich graphs a matched objective ablation finds GraphSAGE at least as strong as a GIN encoder at the fixed budgets the cascade uses, so we adopt it as the homogeneous-graph encoder. OGBN-MAG is a *heterogeneous* academic graph (paper, author, institution, and field-of-study nodes joined by typed relations); we flatten it to a single node/edge index for the pipeline but *retain* node types and relation (edge) types and model them with a relation-aware six-layer RGCN [4] (RGCNConv with per-relation bases over $2R$ relations after symmetrization). Because OGBN-MAG supplies input features only on paper nodes, non-paper nodes receive node-type one-hot and per-relation-degree features, so the retriever reasons over typed structure rather than discarding it. Both encoders use the same retrieval interface: cached fine and coarse partition embeddings are compared to query embeddings, and positive partitions can be live-reencoded during training so coverage gradients update required positives instead of treating the entire partition bank as stale constants.

Training uses three complementary signals: multi-positive contrastive learning pulls a query toward all coarse partitions touched by the planted match; hard negatives keep nearby but incorrect partitions separated; and the FullCov-oriented coverage term penalizes the weakest required positive. For MAG, the production checkpoint is selected by validation FullCov rather than by training epoch; the final-epoch checkpoint is retained only as a separate diagnostic and is never averaged into production.

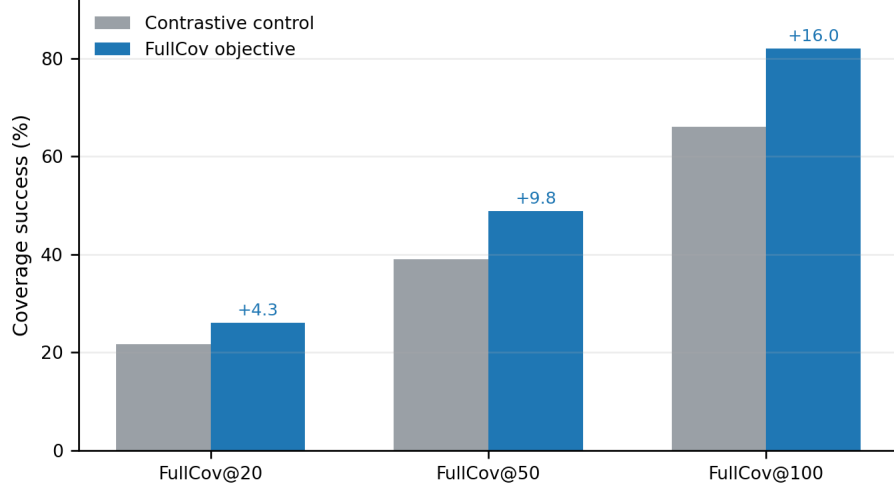


Figure 4: FullCov objective gain. Under matched Arxiv K-hop training conditions, directly optimizing weakest-positive coverage improves every fixed retrieval budget used by the downstream cascade.

C Partition Balance and Overlap Scale

The experiments use the hierarchy below. For a selected partition p , the overlap operator adds only boundary nodes from other partitions that share an edge with p , rather than adding the entire bordering partitions. On MAG, a median partition borders 948 of the 2,000 coarse partitions, but it admits only about 4,252 boundary nodes. The expanded region for one selected partition is therefore about 5,228 nodes (a $5.4\times$ expansion over the 969-node partition), not the $\approx 921K$ obtained by taking every bordering partition in full. That 921K figure is a loose reachability bound the operator never realizes. Candidate size grows only when the cascade unions boundary overlap from hundreds to a thousand retrieved partitions. Because many boundary nodes are shared, the union saturates between 59% and 83% of MAG at the largest budgets (59% for Jigsaw, up to 83% for uninformed policies). Overlap is tight for one partition but collectively broad across a large retrieval budget.

Table 4: Partition balance. Coarse partitions are ranked by retrieval; fine partitions support local overlap and bridge construction. Node counts are per partition.

Dataset	Level	Graph nodes	Parts	Min	Median	Mean	P90	Max
Cora	Coarse	19,793	20	962	986	989.6	1,018	1,019
Cora	Fine	19,793	100	190	197	197.9	204	205
Arxiv	Coarse	169,343	200	822	846	846.7	872	872
Arxiv	Fine	169,343	1,000	153	169	169.3	175	195
MAG	Coarse	1,939,743	2,000	876	969	969.9	999	999
MAG	Fine	1,939,743	10,000	170	194	194.0	200	210

D Query-Family Breakdown and Budget Curves

The query-family view explains the aggregate rates. At the matched half-partition budget the *same* family pattern holds across all three datasets: single and multi-fine saturate, local K-hop families remain high but not perfect on MAG (K-hop 93.3%, degree K-hop 92.7%), and random-walk and multi-coarse are hardest everywhere (random-walk 75.3/82.3/71.0%, multi-coarse 94.3/78.7/76.0% on Cora/Arxiv/MAG). These hard families arise from the query type rather than graph scale. MAG preserves high single (98.7%) and local K-hop recovery, and its multi-region families also solve well (multi-fine 100%, multi-coarse 76.0%); the one family that stays hardest is random walks (71.0%).

Table 5: Overlap expansion scale. “+Boundary overlap” is the median number of boundary nodes the operator actually admits for one selected partition; “Expanded part” is partition plus boundary overlap (with the multiple over the bare partition). “Neighbor parts” counts the partitions a part borders. “Loose union bound” is the median size obtained by taking every bordering partition in full. It is a reachability bound the operator never realizes. The realized per-partition overlap is small ($5.4\times$ on MAG); breadth arises only from unioning over the retrieval budget.

Dataset	Coarse parts	Partition nodes (med)	+Boundary overlap (med)	Expanded part (med / $\times$)	Neighbor parts med / max	Loose union bound (med)
Cora	20	986	534	1,533 / $1.6\times$	18 / 19	18,820
Arxiv	200	846	1,606	2,470 / $2.9\times$	135 / 173	115,004
MAG	2,000	969	4,252	5,228 / $5.4\times$	948 / 1,999	921,494

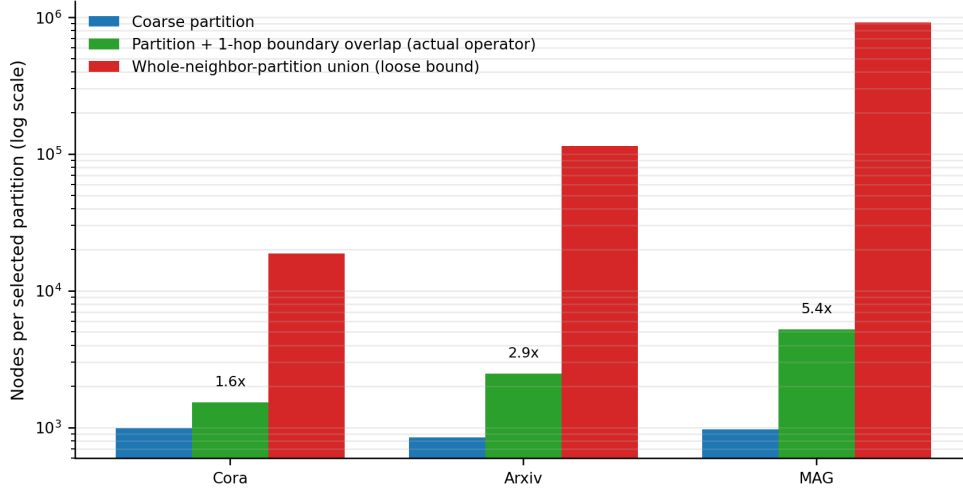


Figure 5: What one selected partition actually contributes (log scale). The realized one-hop boundary overlap (green) expands a coarse partition only modestly ($1.6\times$ on Cora, $2.9\times$ on Arxiv, $5.4\times$ on MAG). The red bar is the loose union-of-whole-neighbor-partitions bound that the operator never realizes; reporting it as “the overlap” overstates MAG breadth by more than $170\times$. Candidate breadth instead comes from unioning these tight per-partition overlaps across the retrieval budget.

486 Random-walk queries provide the clearest stress test. FilterAll solves them at 96.0%, and 96.6%
 487 of Jigsaw’s random-walk misses occur before Glasgow because the planted match never enters the
 488 candidate. Partition count alone does not explain this; random walks span about as many coarse
 489 partitions as K-hop queries (median 29 vs. 23). The difference is overlap recovery. A K-hop query
 490 forms a compact BFS ball, so nodes in a missed partition often lie on the boundary of a retrieved one
 491 and one-hop overlap recovers them 94% of the time. A random walk is a diffuse path whose missed
 492 mid-path partitions are less often adjacent to a retrieved boundary, so overlap recovers 62% of misses.
 493 Recovery changes from 0.94 at size 20 to 0.40 at size 100. The effect is milder on the smaller, denser
 494 Arxiv, where random-walk recovery is 82.3% at the half-partition budget and reaches 100% at all
 495 200 partitions. On MAG, the learned retriever leads Mean-RRF by 10 points for this family and by
 496 15 points at size 100, showing that better ranking is the effective lever once overlap cannot bridge the
 497 footprint.

498 MAG’s high-degree quotient graph explains why ranking is the decisive mechanism. A coarse
 499 partition borders a median 948 of 2,000 others, and a fine partition borders a mean 568 of 10,000, so
 500 broad neighborhood expansion cannot selectively recover a missed mid-walk region. The inference-
 501 time probes in Appendix G reinforce this diagnosis and identify footprint-aware representation
 502 learning as the next target. The deployed MAG encoder addresses the regime during training with a
 503 random-walk- and multi-coarse-weighted distribution of 50 to 100-node queries. Its training stream
 504 (seed 7203) is disjoint from held-out evaluation seeds 20260607/08. Relative to a generic-mix encoder
 505 (86.0% overall, random-walk 60.0%, multi-coarse 71.7%), walk-aware training lifts random-walk
 506 to 71.0% and multi-coarse to 76.0%. Overall MAG recovery reaches **88.6%** while preserving the

Table 6: Jigsaw query-family breakdown. Positive columns report exact-solve rate; negative columns report correct no-match rate. All three datasets use the corrected connected multi-family reruns. All datasets are reported at the matched half-partition budget ($K=10/100/1000$); the MAG rows use the deployed walk-aware encoder (overall 88.6%).

Dataset	Single	K-hop	Deg-K	Walk	Multi-fine	Multi-coarse	Neg-label	Neg-struct
Cora	100.0	98.3	97.3	75.3	100.0	94.3	100.0	100.0
Arxiv	100.0	97.3	98.3	82.3	100.0	78.7	100.0	100.0
MAG	98.7	93.3	92.7	71.0	100.0	76.0	100.0	99.0

easy-family results. Across all 1800 positives, walk-aware Jigsaw beats Mean-RRF 92 to 35 (exact $p=4.4 \times 10^{-7}$); the advantage is concentrated in multi-coarse (34/12) and random-walk (44/13), while the remaining families are statistically tied (14/10).

The released cumulative solved-by-budget summaries make the budget dependence explicit. With the overlap-aware retrievers, Jigsaw’s cumulative solve rises 64.0/82.7/94.2/100 on Cora (budgets 2/5/10/20), 69.6/81.3/92.8/100 on Arxiv (20/50/100/200), and 37.6/46.0/53.0/61.9/75.4/88.6 on MAG (20/50/100/200/500/1000) with the deployed walk-aware encoder; read at the half-partition budget Jigsaw solves 94.2/92.8/88.6 on Cora/Arxiv/MAG. The 100% Cora/Arxiv figures require every partition and are the exhaustive ceiling. MAG’s per-size slices at $K=1000$ are 93.2/90.7/82.0 for sizes 20/50/100.

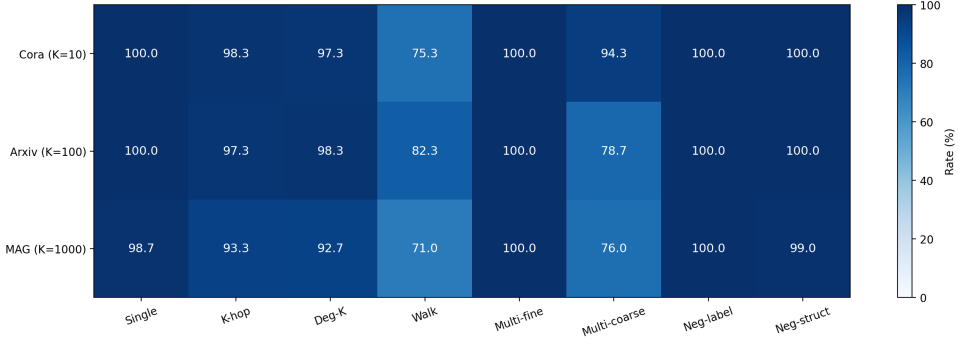


Figure 6: Jigsaw query-family outcomes. Positive families report exact-solve rate; negative families report correct no-match rate. All rows are at the matched half-partition budget ($K=10/100/1000$). Random-walk and multi-coarse are the hard families across *all three* datasets (Walk 75/82/71%, Multi-coarse 94/79/76%); single and multi-fine templates saturate, while local K-hop variants remain high but not perfect on MAG.

On Cora the curve rises steeply (64.0% at budget 2 to 100% by budget 20), so a handful of partitions decides most queries. Arxiv needs budget expansion to 200 before reaching its final positive solve rate, even though all policies eventually reach 100%. MAG continues to improve through 500 and 1000, which means failures at small budgets are often not verifier unsoundness but insufficient candidate expansion. This supports exposing budget as a control knob rather than using one fixed candidate size for all queries.

E Expanded MAG Results and Statistical Significance

This appendix gives the per-family, cost, and significance details behind the condensed MAG discussion and Table 2. The reported aggregates are the same as those in the body.

Matched Cora/Arxiv cost and difficulty. At the same half-partition endpoints used for solve rate, Jigsaw sends a mean 458/77 terminal candidate nodes on Cora/Arxiv and completes the cascade in 0.05/0.19 s (11.8/18.3 ms in the verifier). Cora size-100 recovery is lower and costlier than sizes 20/50 (89.0% vs. 96.8/96.8%; 0.071 s vs. 0.036/0.047 s), driven by random-walk (75.3%). Arxiv

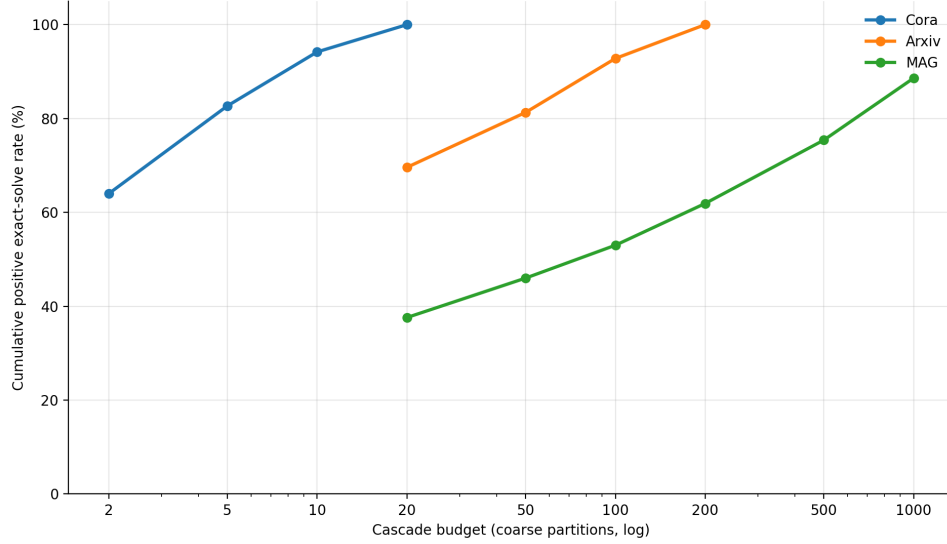


Figure 7: Solved-by-budget curves for Jigsaw. Cora solves early because its candidate graphs are small and labels are strong. Arxiv benefits from budget expansion through 200. MAG continues improving through 1000, reflecting larger candidate regions and harder coverage.

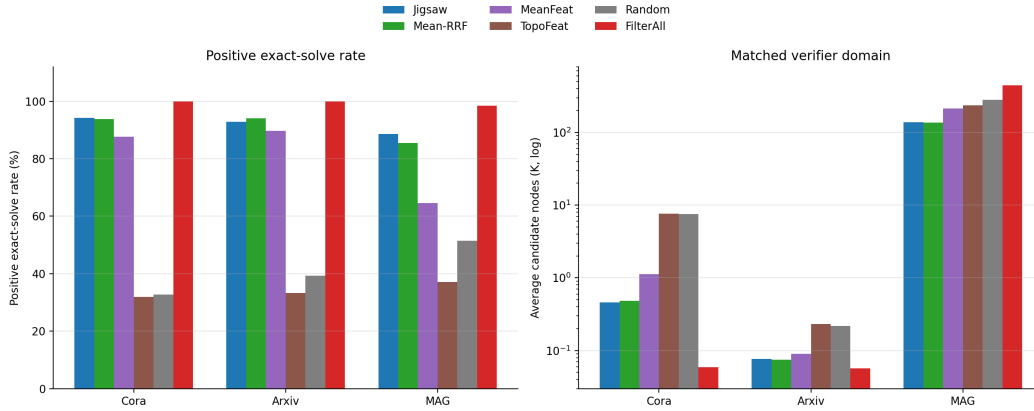


Figure 8: Production positive solve rates and matched cost. Left: positive exact-solve rate at half-partition budgets ($K=10/100/1000$); FilterAll is the exhaustive ceiling. Right: mean pruned candidate nodes at those same endpoints (FilterAll at the all-partition ceiling).

530 similarly declines to 90.3% at size 100 from 93.8/94.2%, with multi-coarse hardest (78.7%); single
 531 and multi-fine remain 100% on both graphs. FilterAll selects every partition, but exact-label pruning
 532 reduces its terminal verifier domain to only 59/57 nodes. Its exhaustive cost comes from retrieval
 533 scope and residence, not a large post-pruning graph.

534 **The MAG lower bound and scaling crossover, in full.** Direct Glasgow solves 0/15 full-MAG
 535 K-hop queries and 0/15 on Arxiv, versus 15/15 on Cora. On those same Cora/Arxiv queries, Jigsaw at
 536 matched half-partition budgets solves 15/15 in median 0.03/0.11 s with about 50 pruned nodes. The
 537 Cora cascade result therefore does not rely on retrieving all 20 partitions. FeatureIndex is strongest
 538 under near-unique MAG labels, FilterAll is the recall ceiling, and learned retrieval is useful under
 539 bounded memory and coarse labels. On MAG, Jigsaw uses fewer candidate nodes than FilterAll
 540 (138K versus 444K) and lower verifier time (82 ms versus 289 ms), although candidate construction
 541 remains the dominant cost.

Statistical significance. Bootstrap 95% confidence intervals on the MAG positive solve rate (n=1800 queries per method) and paired exact-McNemar tests (pairing per query so wins/losses count only queries two policies decide differently) confirm the ordering. Among the retrieval components, the FullCov-trained retriever significantly outperforms the other learned and feature retrievers (Mean-RRF, re-evaluated under the same walk-aware encoder, 92 vs 35 wins, $p=4.4 \times 10^{-7}$; mean features / random / topology $p < 10^{-76}$), and FilterAll significantly exceeds it (225 vs 2 wins, $p=2.4 \times 10^{-64}$) as the exhaustive ceiling; the latter four are encoder-independent and reported against the base-encoder Jigsaw. These comparisons evaluate retrieval components within the same system. The paper’s contribution is exact matching at million-node scale without holding the graph resident. The retriever keeps the candidate both small and memory-bounded, sending the verifier $3.2 \times$ fewer candidate nodes than the exhaustive FilterAll ceiling, and FullCov improves coverage on the hard families relative to the matched learned/fusion baselines.

Candidate construction is volume-bound. Mean per-query candidate-build time is 3.4 s for Jigsaw and 5.5 s for FilterAll, compared with 0.25 s for exact solving (Section 6.3, fact 3). These means come from the generic-encoder shrinkage diagnostic. The production-matrix cascade times use the walk-aware encoder and also exclude retrieval/ranking, but the two runs are not directly additive. Node volume, not the union implementation, sets the build cost: a verified-equivalent vectorized tensor union runs between 0.5 and $0.9 \times$ as fast as the set union. The practical latency lever is therefore better localization, which reduces the number of boundary nodes admitted before exact verification.

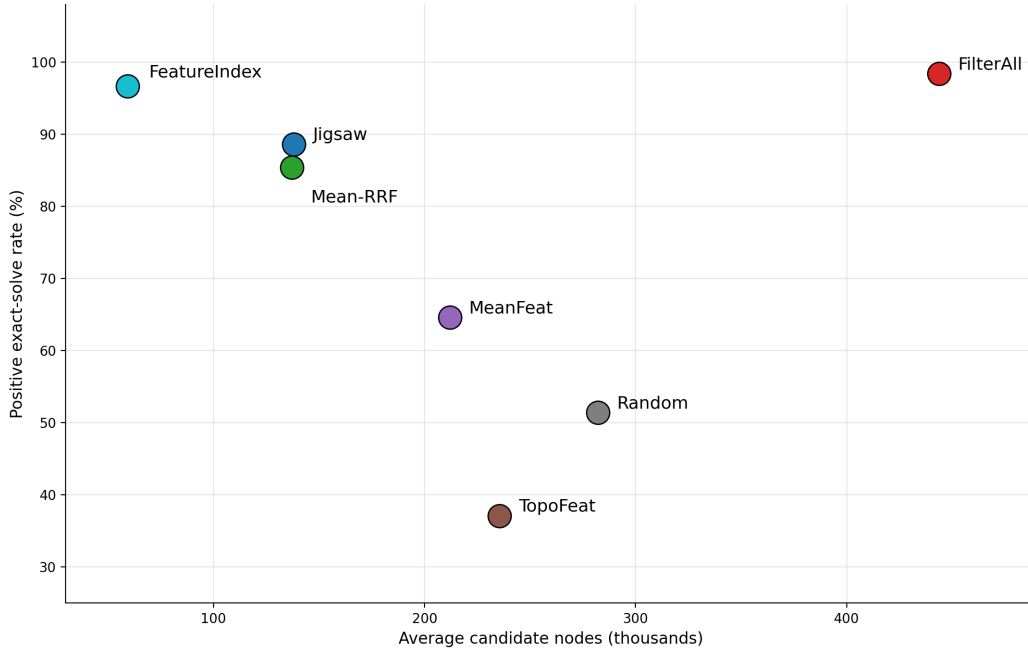


Figure 9: MAG tradeoff. MAG is the production dataset where retrieval policies separate sharply. FeatureIndex is strongest under near-unique labels; FilterAll is the exhaustive ceiling; learned Jigsaw is a memory-bounded operating point that uses less than half the candidate nodes of FilterAll.

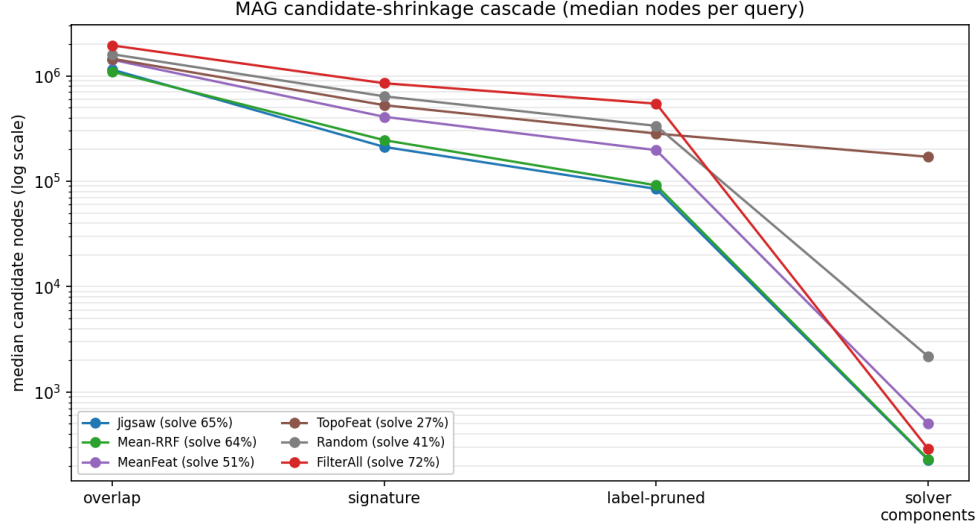


Figure 10: MAG candidate-shrinkage cascade (median nodes per query, log scale). Each policy’s candidate is tracked through overlap, signature, label pruning, and component decomposition. Learned retrieval (Jigsaw) and Mean-RRF reach much smaller solver inputs than uninformed policies, and far smaller than FilterAll, even though all share the same lossless pruning.

F System-Design Ablations and Model-Variant Diagnostics

The operators remain dataset-dependent, but the matched-budget comparison removes the full-budget confound. On MAG, removing one-hop overlap halves solve rate (88.9→44.4%), while signatures are inert (88.9% and about 151K candidates with or without them). On Arxiv, overlap has a smaller but real effect (94.4→86.1%). Signatures are the main candidate-control mechanism there, expanding the mean pruned domain $\sim 37\times$ (76→2,815) when removed. Exact-label pruning reduces Arxiv candidates (244→76) and remains important for negatives; removing it lowers MAG solve rate (88.9→52.8%). Component decomposition also helps MAG (88.9→63.9% without it). Together with the Arxiv retrieval ablation in Table 1, these results separate the learning gain from the execution gains supplied by overlap, pruning, and component solving.

Table 7: Operator-necessity ablation at matched half-partition budgets. The shared diagnostic has 48 queries per dataset; solve and candidate statistics use its 36 planted positives. We report MAG at $K = 1,000/2,000$ and Arxiv at $K = 100/200$. Candidate count is the mean pruned domain at the first solved budget at or below K , or the largest attempted budget at or below K when unsolved.

Variant	OGBN-MAG		OGBN-Arxiv	
	Solve %	Cand.	Solve %	Cand.
Full Jigsaw	88.9	150.6K	94.4	76
No overlap	44.4	152.5K	86.1	88
No signature	88.9	150.7K	94.4	2,815
No components	63.9	143.0K	94.4	76
No exact-label	52.8	223.6K	94.4	244

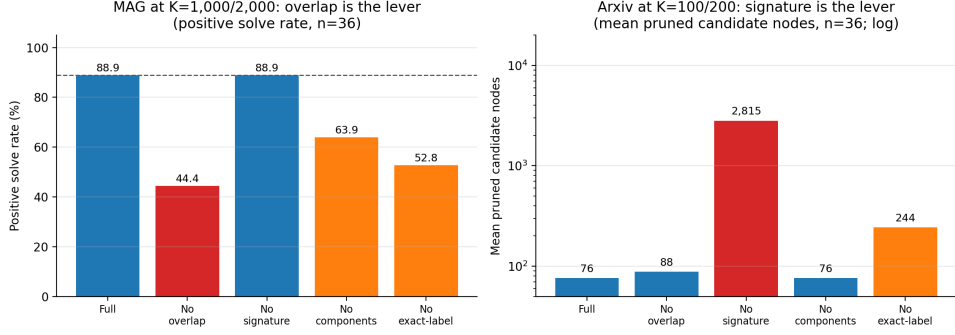


Figure 11: Operator necessity at matched half-partition budgets. *Left:* on MAG, disabling overlap halves solve rate (88.9→44.4%). *Right:* on Arxiv, disabling signatures expands mean pruned candidates $\sim 37\times$ (76→2,815, log axis); removing overlap lowers solve rate (94.4→86.1%).

572 **Model-variant diagnostics.** We retain model-variant diagnostics outside the main benchmark.
 573 On Arxiv, extended training changes retrieval behavior, but the reported result uses the checkpoint
 574 selected before evaluation in a balanced two-seed matrix. On MAG, we evaluated both the validation-
 575 selected and final-epoch checkpoints. The main result uses the former because validation FullCov
 576 selected it; the latter is reported separately and is not averaged into the benchmark.

577 G Retrieval Remedies and Cross-Dataset Selectivity

578 **The label-selectivity crossover generalizes.** Repeating the retrieval comparison on Cora and
 579 OGBN-Arxiv (retrieval-only, identical query families, $n=540$ each) reproduces the MAG inversion
 580 (Table 8). It tracks the median number of partitions spanned by a label, an offline statistic (Cora
 581 feature labels 1, class 12; Arxiv feature 1, class 124.5). Under near-unique feature labels the classical
 582 index ties the learned retriever on contained families and wins overall (Cora rank 2 vs. 5; Arxiv 4
 583 vs. 44); under realistic class labels it collapses toward random (Arxiv 104 of 200) while the learned
 584 retriever wins every family (Cora 5 vs. 7; Arxiv 44 vs. 104; per-query sign test $p < 10^{-30}$ in both
 585 regimes, $n=540$). Selecting the retriever by this offline signal is a deployable, non-oracle portfolio;
 586 on Cora/Arxiv it is a localization (retrieval-rank) result, corroborated end-to-end only on MAG.

Table 8: Cross-dataset label-selectivity crossover: overall median worst-true-partition rank (lower is better; retrieval-only, $n=540$ per dataset = 2 seeds $\times$ 6 positive families $\times$ 3 sizes $\times$ 15 queries).

Dataset (parts)	FeatureIndex		Learned (any)	Parts/label feat / class
	feature	class		
Cora (20)	2	7	5	1 / 12
Arxiv (200)	4	104	44	1 / 124.5

587 **Spatially extended queries require representation-level coverage.** On MAG, subquery multi-
 588 vector MaxSim, finer partition parents, one- and two-hop fine-level overlap, dual-granularity late
 589 interaction, and coarse diffusion all leave FullCov@1000 within ≈ 2 points of the single-vector
 590 baseline on the k-hop/degree-k-hop/random-walk families (Table 9); the best gain is $\approx +2.3$ (coarse
 591 diffusion), while a coarse neighbor-stitch expansion reduces coverage by 8 to 13 points. The cause
 592 is density: the partition graph is high-degree at every granularity (median coarse degree 948 of
 593 2,000; mean fine degree 568 of 10,000), so neighborhood-following broadens the candidate rather
 594 than tracing the query footprint. This evidence directs further improvement toward coverage-aware
 595 representation learning.

Table 9: Inference-time remedies on the same 1,800-query MAG positive workload: FullCov@1000 (%). Each displayed spatial family has $n=300$ queries.

Method	k-hop	degree-k-hop	random-walk
Single-vector (current)	14.7	17.3	9.0
Subquery MaxSim (query-only)	14.3	17.7	8.0
Finer partition parent	12.7	16.7	8.3
Fine-level overlap (1-hop)	13.3	17.3	10.0
Fine-level overlap (2-hop)	13.3	18.3	9.3
Dual-granularity late interaction	12.7	16.0	8.7
Coarse diffusion	17.0	19.7	10.3

H Memory-Bounded Streaming Serve

The streaming serve holds only an LRU cache of partitions resident and fetches the rest from disk in a two-pass sweep (nodes, then edges), so peak resident memory is bounded by the cache size rather than by the graph. The conservative primary serve reported in the body uses 2.4 GB versus 10.2 GB for whole-graph residence ($4.2\times$ smaller). Figure 12 additionally reports an optimized cache sweep whose eight-partition point is 1.9 GB. Both implementations make memory a tunable resource; their cold-load latency is competitive and load-dominated, and larger caches buy down tail latency.

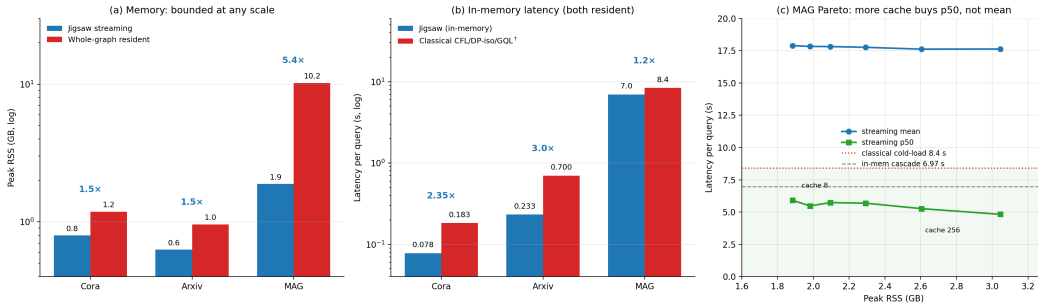


Figure 12: Memory is the primary benefit; cold-load latency is competitive and the tradeoff is tunable. (a) The conservative primary streaming serve is $4.2\times$ smaller on MAG (2.4 vs 10.2 GB); the optimized cache sweep shown here lowers its eight-partition point to 1.9 GB ($5.4\times$). (b) Cold-load end-to-end latency, Jigsaw vs. classical CFL/DP-iso/GQL: $2.35/3.0/1.2\times$ faster on Cora/Arxiv/MAG because Jigsaw avoids the full-graph load († MAG resident enumeration is ~ 1 ms after loading). (c) MAG memory $\leftrightarrow$ latency frontier as the cache grows (8 $\rightarrow$ 256): more memory buys down p50 (5.9 $\rightarrow$ 4.8 s, below the 8.4 s cold-load) while the mean remains 17.9 to 17.6 s because of a hard-query I/O tail. Jigsaw lets the operator choose a frontier point an all-or-nothing resident matcher cannot.

Generative AI Usage Statement

Generative AI tools assisted with implementation, experiment orchestration, data-validation scripts, and manuscript language editing. The authors reviewed and verified all AI-assisted code, analyses, numerical claims, and text, and take full responsibility for the final work.